3D SHAPE REPRESENTATION LAB

POINT CLOUDS · VOXELS · MESHES

EXECUTION REPORT | SECTIONS 7.2-7.5

A practical implementation and evidence report covering PointNet++, mesh voxelization, MeshCNN, and cross-representation conversions.

J GANESH KUMAR REDDY

Student ID 23BDS024

Submission date 30 October 2026

PointNet++ on ModelNet40 / Mesh-to-voxel CNN / MeshCNN on SHREC / Representation conversion

1. Executive summary

This practical report executes the main code paths described in Sections 7.2-7.5: point-cloud classification with the upstream PointNet++ implementation, mesh voxelization and a 3D CNN, edge-based MeshCNN processing, and conversions among meshes, points, and voxels.

Important scope note. The official ModelNet40, ModelNet10, and SHREC archives were not available in the execution environment. The full 200-epoch ModelNet40 benchmark and SHREC training were therefore not run. All reported successful runs use explicitly generated synthetic fixtures and are smoke tests only; their accuracy/loss values are not benchmark results.

Execution status

Section	Execution	Evidence / scope
7.2 PointNet++	**BLOCKED**	200-epoch run stopped before training: upstream loader could not find the official ModelNet40 metadata. Upstream SSG code then passed a one-epoch synthetic CPU smoke test.
7.3 Voxel CNN	**PASS - SMOKE**	Generated cube OFF mesh -> 32 x 32 x 32 occupancy -> untrained 10-class CNN output shape (1, 10).
7.4 MeshCNN	**PASS - SMOKE**	Upstream edge-convolution path, 42-vertex / 80-triangle synthetic icosphere, one optimizer step; no SHREC data.
7.5 Conversions	**PASS**	4,096 area-sampled points, SciPy convex-hull mesh, 32-cubed occupancy, and occupied-cell center cloud.

Result in one sentence

The software pathways were exercised end-to-end on small fixtures, while benchmark-level claims remain deferred until the official datasets and suitable hardware are supplied.

2. Environment, source and data access

Runtime used for the captured execution

Component	Observed runtime	Notes
Python	3.11.2	Base sandbox interpreter; Python 3.7 / 3.6 environments from the original instructions were not used.
PyTorch	2.2.2 (CUDA 12.1 build)	CUDA runtime is packaged, but no CUDA device is visible; all tested models ran on CPU.
Compute	2 visible CPU cores; 3.85 GiB host memory	CPU smoke tests only; the requested full training schedule was not attempted on this device.
Geometry/report	NumPy 1.26.4; SciPy 1.11.4; trimesh 4.5.3; Matplotlib 3.8.4; ReportLab 4.2.5	Exact pins are listed in requirements.txt.

Upstream implementations pinned for this run

Repository	Commit used
yanx27/Pointnet_Pointnet2_pytorch	eb64fe0b4c24055559cea26299cb485dcb43d8dd
ranahanocka/MeshCNN	5bf0b899d48eb204b9b73bc1af381be20f4d7df1

Dataset availability

A HEAD request to the ModelNet40 Stanford host returned an SSL connection error; the ModelNet10 Princeton HTTP endpoint returned an empty response. The MeshCNN SHREC helper points to a Dropbox archive, which was not fetched. As a result, the official data could not be prepared within this sandbox. The full PointNet++ attempt confirms the missing metadata at the loader boundary.

Fixture policy

The script `prepare_fixtures.py` creates a 40-name ModelNet-style metadata layout with two synthetic training clouds and one synthetic test cloud per class, a small cube OFF file, and a triangular icosphere OBJ. These are deliberately marked as synthetic and must not be confused with the named public benchmarks.

3. Section 7.2 - PointNet++ on ModelNet40

PointNet++ builds hierarchical point features through set-abstraction stages: farthest-point sampling chooses centers, ball query forms local neighborhoods, pointwise multilayer perceptrons encode local geometry, and max pooling aggregates each group. The SSG classifier reduces the hierarchy to a global descriptor and a 40-way classification head.

What was run

The unmodified upstream `train_classification.py` and `pointnet2_cls_ssg` model were executed on CPU. For the smoke test, the synthetic ModelNet-style split contains 2 training examples in 2 classes and 40 test examples (one per output class), with 512 points per cloud. Training was limited to one epoch and one batch. Normals and uniform sampling were disabled, matching the upstream defaults.

```
python train_classification.py \
  --model pointnet2_cls_ssg \
  --log_dir pointnet2_synthetic_smoke \
  --num_category 40 --epoch 1 --batch_size 2 \
  --num_point 512 --use_cpu
```

Observed outcome

Measure	Observed smoke-test value	Interpretation
Training	1 epoch; 2 synthetic examples; train instance accuracy 0.000	Only confirms that a training update completed; it is not a learning curve.
Internal test pass	40 synthetic examples; instance accuracy 0.025; class accuracy 0.025	Sanity output on generated data only. The 2.5% result is not ModelNet40 accuracy.
Requested benchmark	200 epochs; official normal-resampled ModelNet40	Not run: missing official metadata/data archive.

The 200-epoch command exited at dataset initialization with `FileNotFoundError: data/modelnet40_normal_resampled/modelnet40_shape_names.txt`. No official training examples were processed.

CPU test caveat

The training script's internal validation loop completed. The separate upstream `test_classification.py` contains an unconditional CUDA allocation for its vote pool, so standalone CPU-only testing is not supported by that script without a small upstream change.

4. Section 7.3 - Volumetric data and 3D CNN

The voxel demonstration loads a closed OFF cube, centers and isotropically scales it, voxelizes at unit pitch, fills the interior, and places occupancy into a fixed 32-cubed grid. The input tensor is arranged as batch, channel, depth, height, width before the 3D convolutions.

Executed demonstration

Item	Result	Meaning
Fixture mesh	Synthetic cube OFF; 12 triangles after loading	A local stand-in, not a downloaded ModelNet10 chair.
Occupancy grid	32 x 32 x 32; 24,389 / 32,768 cells occupied (74.43%)	Filled occupancy volume; aspect ratio preserved and grid is fixed-size.
CNN forward	Input (1, 1, 32, 32, 32); logits (1, 10)	Two Conv3D/MaxPool blocks followed by 32768-to-256-to-10 fully connected layers.
Prediction	Index 7 from an untrained head	A shape/forward-pass check only; not a class prediction result.

Memory scaling

A float32 occupancy tensor alone uses 4 bytes per cell. Thus 32-cubed is 32,768 cells (0.125 MiB), 64-cubed is 262,144 cells (1 MiB), and 128-cubed is 2,097,152 cells (8 MiB). Convolution activations, intermediate feature maps, batches, and gradients add substantially more memory.

The notebook-style demo was executed as a single CPU forward pass. ModelNet10 archive loading and any training/evaluation of the 3D CNN were not performed.

5. Section 7.4 - MeshCNN on SHREC

MeshCNN convolves over each mesh edge and its four one-ring edge neighbors. Its input descriptors include dihedral angle, symmetric opposite-face angles, and symmetric geometric ratios; the mesh connectivity determines which neighboring features are gathered. This preserves topology that an unstructured point set or dense occupancy grid does not explicitly store.

Executed upstream-code smoke test

The upstream MeshCNN mesh preparation and MeshConv classifier were loaded from the pinned repository. A generated icosphere (42 vertices, 80 triangular faces, 120 edges) produced 5 x 120 edge features. A compact single-stage MeshCNN classification head returned 30 logits, then ran one cross-entropy backward pass and Adam update on a synthetic label.

Check	Observed	Limit
Edge feature shape	(5, 120)	Generated icosphere only.
Classifier output	(1, 30)	30-way head configured for a smoke shape; no SHREC labels were loaded.
Optimization	Single synthetic-label loss 3.177232; backward and Adam step passed	Loss is not an evaluation metric and is not comparable to SHREC results.

Compatibility note. The 2019 repository targets an older NumPy release. The smoke runner applied process-local compatibility shims for NumPy's ragged per-vertex edge list serialization and the removed `np.bool` alias; the cloned upstream source files were not edited.

Not completed

The SHREC16 archive and pretrained weights were not fetched; the upstream setup expects an older Python/PyTorch environment. Therefore `scripts/shrec/train.sh`, `test.sh`, and `view.sh` were not run, and no SHREC accuracy is reported.

6. Section 7.5 - Converting between representations

Four conversions were executed on the same generated cube, keeping the source geometry fixed so that the representation changes are easy to compare.

Conversion	Method used	Observed output
Mesh to point cloud	trimesh area-uniform surface sampler	4,096 points; array shape (4096, 3).
Point cloud to mesh	SciPy ConvexHull	106 hull vertices; 208 triangular faces.
Mesh to voxel	trimesh voxelize + fill, centered 32-cubed grid	24,389 occupied cells.
Voxel to point cloud	Coordinates of occupied-cell centers	24,389 center points.

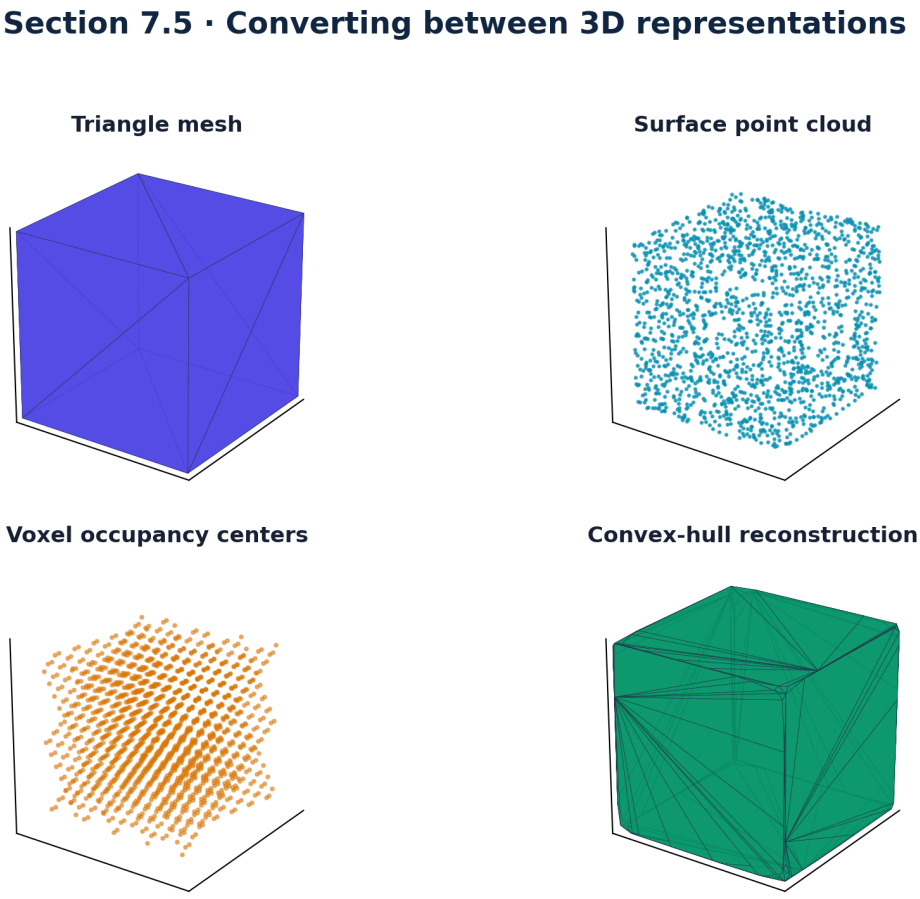


Figure 1. Mesh, sampled surface cloud, occupied voxel centers, and convex-hull reconstruction. The orange panel displays a deterministic subset of voxel centers for readability.

Method limitation. ConvexHull is a deliberate lightweight reconstruction for this convex cube. It is not Poisson reconstruction or Ball Pivoting, and it would erase concavities or cavities in non-convex shapes. For those cases, a normal-aware Poisson/BPA implementation such as Open3D is more appropriate.

7. Findings and interpretation

Representation trade-offs observed

Representation	Strength	Trade-off / risk
Point cloud	Compact surface samples; no fixed grid required; PointNet++ learns local neighborhoods.	Sampling density and point ordering can vary; this run did not validate generalization.
Voxel grid	Regular tensor makes 3D convolution simple; occupancy explicitly represents filled volume.	Resolution quantization and cubic memory scaling; voxelization can thicken surfaces.
Mesh	Retains vertices, faces, and edge adjacency; MeshCNN uses topological neighborhoods.	Requires valid/manifold connectivity, preprocessing, and stable edge pooling.

Key findings

1. **Pipeline checks are not benchmark results.** The upstream PointNet++ train/evaluate loop ran, but only on 42 synthetic examples. The 0.025 test score is the random-level outcome expected in a 40-way toy setup and must not be presented as ModelNet40 performance.

2. **Topology matters.** MeshCNN's edge-convolution path consumed edge adjacency and five local geometric feature channels. Converting the same mesh to only points or occupancy would discard that explicit edge neighborhood.

3. **Conversions are lossy.** Surface sampling discards connectivity; voxelization quantizes coordinates; occupied-cell centers are a discrete point set; convex-hull reconstruction enforces convexity.

4. **Grid scaling is cubic.** Doubling a voxel grid edge length multiplies cell count by eight. The occupancy array is only the input tensor; activation memory is often the real constraint.

5. **Reproducibility needs data + hardware metadata.** A real run should record data release, split files, seed, point count, normals/uniform-sample flags, CUDA/PyTorch versions, checkpoints, and test protocol.

Overall conclusion: Sections 7.2-7.5 are implemented as verified CPU smoke tests and conversion demonstrations. To finish the intended experiments, repeat the runs with the official datasets; use the original benchmark protocols and report real test metrics only after that evaluation.

8. Reproducibility and next steps

Re-run the captured smoke-test bundle

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
./run_all.sh
```

The runner creates synthetic fixtures, checks out the upstream repositories under an ignored external/ folder unless their paths are supplied, executes the one-epoch PointNet++ and one-step MeshCNN smoke runs, runs voxelization/conversions, saves logs and terminal transcript screenshots, and rebuilds this PDF.

To complete the official experiments

1. Install the upstream-compatible PyTorch/CUDA environment for the machine used. The old pinned stacks in the source instructions may require an older Python interpreter; check upstream installation notes before changing versions.

2. Download the official aligned ModelNet40 normal-resampled archive and verify that the root has the three metadata files and one class folder per category. Do not substitute the generated synthetic directory for real data.

```
cd Pointnet_Pointnet2_pytorch
# place official data at data/modelnet40_normal_resampled/
python train_classification.py --model pointnet2_cls_ssg \
  --log_dir pointnet2_modelnet40 --num_category 40 \
  --epoch 200 --batch_size 24
python test_classification.py --log_dir pointnet2_modelnet40
```

3. Obtain the SHREC16 archive using the source repository's current dataset instructions; then run its SHREC train, test, and view scripts in a compatible MeshCNN environment. Record the split, number of classes, training schedule, and test metrics.

4. Re-run the voxel CNN on genuine ModelNet10 .off meshes. For non-convex point-cloud reconstruction, use a normal-aware Poisson or Ball Pivoting implementation; keep the convex-hull demo labelled as such.

Submission contents. This report includes the limitations, measured smoke-test outputs, conversion figure, and eight black-background terminal transcript screenshots. The raw saved logs and generated artifacts are kept alongside the PDF in ganeshlaptop/outputs/.

References

[1] Yan, X. et al. PointNet++ PyTorch implementation. https://github.com/yanx27/Pointnet_Pointnet2_pytorch (revision listed in Section 2).

[2] Hanocka, R. et al. MeshCNN: A Network with an Edge. ACM Transactions on Graphics, 38(4), 2019. https://github.com/ranahanocka/MeshCNN (revision listed in Section 2).

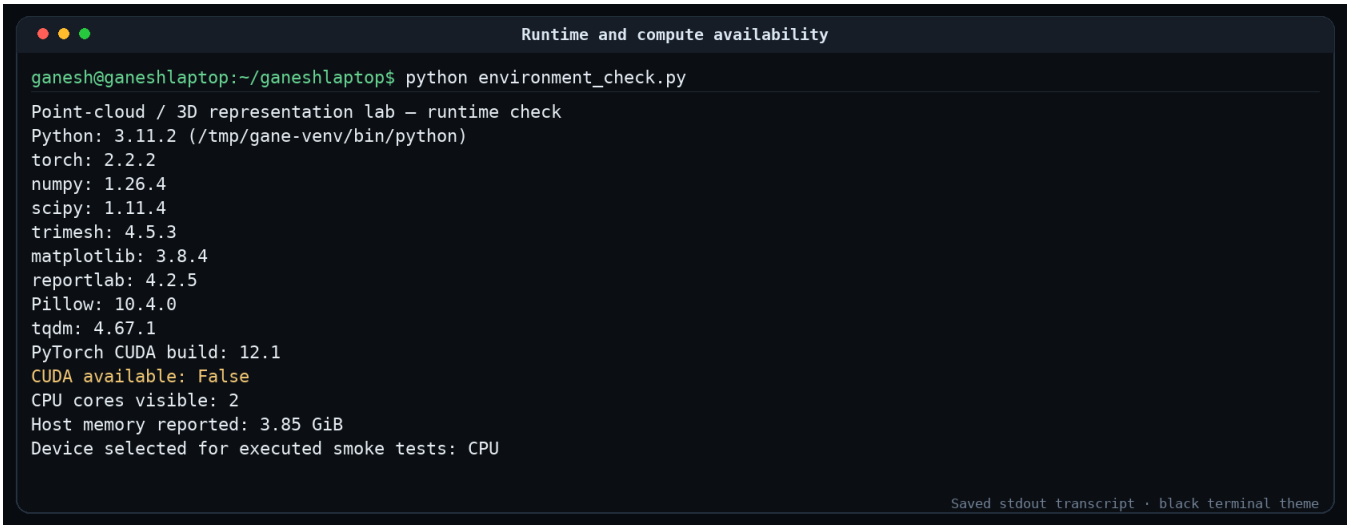
[3] ModelNet40 aligned normal-resampled archive: https://shapenet.cs.stanford.edu/media/modelnet40_normal_resampled.zip. ModelNet10 source location as supplied in the lab brief: http://3dvision.princeton.edu/projects/2014/3DShapeNets/ModelNet10.zip.

[4] MeshCNN repository SHREC scripts and data loader: https://github.com/ranahanocka/MeshCNN/tree/master/scripts/shrec.

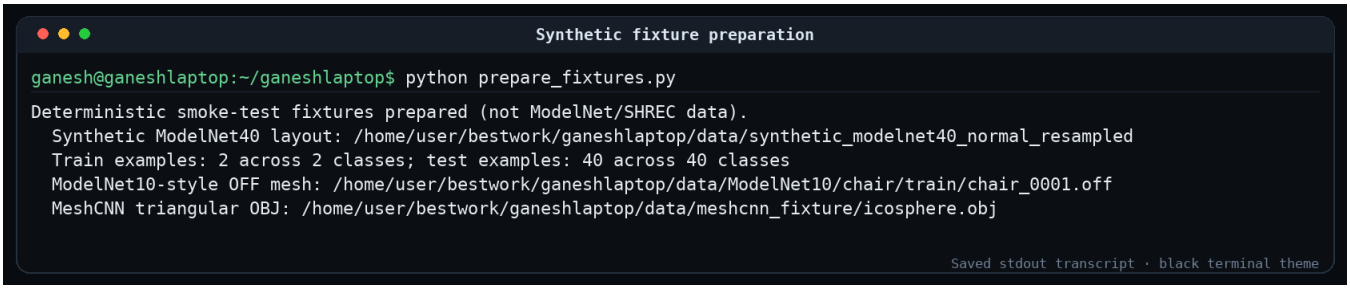
Appendix A - environment and dataset access

Terminal transcript screenshots are rendered on a black terminal theme from the saved command stdout/stderr logs in outputs/logs/. They preserve the actual run messages while removing transient carriage-return progress updates for readability.

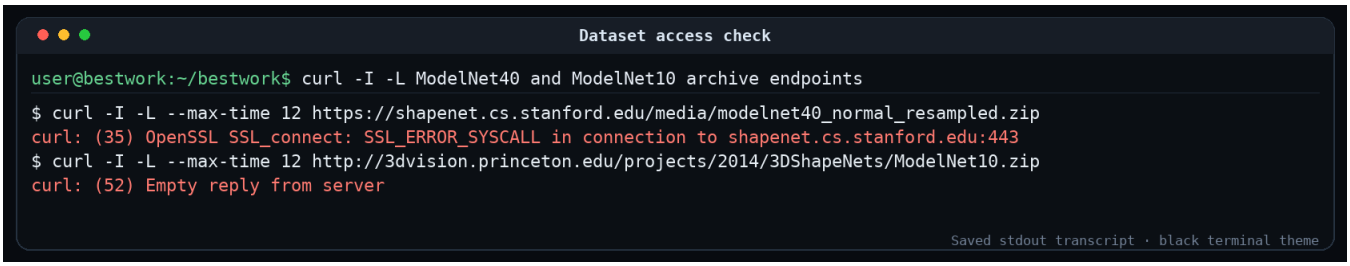
Runtime check



Deterministic smoke-test fixture creation



Dataset endpoint probe (HEAD requests)



Appendix B - PointNet++ execution

Terminal transcript screenshots are rendered on a black terminal theme from the saved command stdout/stderr logs in outputs/logs/. They preserve the actual run messages while removing transient carriage-return progress updates for readability.

Full 200-epoch attempt: failure at missing official metadata

PointNet++ full-run attempt (dataset missing)

```
user@pointnet2:/tmp/pointnet-upstream$ python train_classification.py --model pointnet2_cls_ssg --log_dir
  pointnet2_modelnet40 --num_category 40 --epoch 200 --batch_size 24 --use_cpu
PARAMETER ...
Namespace(use_cpu=True, gpu='0', batch_size=24, model='pointnet2_cls_ssg', num_category=40, epoch=200,
    learning_rate=0.001, num_point=1024, optimizer='Adam', log_dir='pointnet2_modelnet40', decay_rate=0.0001,
    use_normals=False, process_data=False, use_uniform_sample=False)
Load dataset ...
Traceback (most recent call last):
  File "/tmp/pointnet-upstream/train_classification.py", line 232, in <module>
    main(args)
  File "/tmp/pointnet-upstream/train_classification.py", line 121, in main
    train_dataset = ModelNetDataLoader(root=data_path, args=args, split='train', process_data=args.process_data)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/tmp/pointnet-upstream/data_utils/ModelNetDataLoader.py", line 63, in __init__
    self.cat = [line.rstrip() for line in open(self.catfile)]
                                          ^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory:
    'data/modelnet40_normal_resampled/modelnet40_shape_names.txt'
EXIT_CODE=1
```

Saved stdout transcript · black terminal theme

One-epoch upstream-code synthetic CPU smoke run

PointNet++ official-code CPU smoke run

```
user@pointnet2:/tmp/pointnet-upstream$ python train_classification.py --model pointnet2_cls_ssg --log_dir
  pointnet2_synthetic_smoke --num_category 40 --epoch 1 --batch_size 2
  --num_point 512 --use_cpu
PARAMETER ...
Namespace(use_cpu=True, gpu='0', batch_size=2, model='pointnet2_cls_ssg', num_category=40, epoch=1,
    learning_rate=0.001, num_point=512, optimizer='Adam', log_dir='pointnet2_synthetic_smoke',
    decay_rate=0.0001, use_normals=False, process_data=False, use_uniform_sample=False)
Load dataset ...
The size of train data is 2
The size of test data is 40
No existing model, starting training from scratch...
Epoch 1 (1/1):
100%|██████████| 1/1 [00:00<00:00, 2.12it/s]
100%|██████████| 1/1 [00:00<00:00, 2.01it/s]
Train Instance Accuracy: 0.000000
100%|██████████| 20/20 [00:02<00:00, 8.10it/s]
100%|██████████| 20/20 [00:02<00:00, 6.85it/s]
Test Instance Accuracy: 0.025000, Class Accuracy: 0.025000
Best Instance Accuracy: 0.025000, Class Accuracy: 0.025000
Saving at log/classification/pointnet2_synthetic_smoke/checkpoints/best_model.pth
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme

Appendix C - voxel and MeshCNN execution

Terminal transcript screenshots are rendered on a black terminal theme from the saved command stdout/stderr logs in outputs/logs/. They preserve the actual run messages while removing transient carriage-return progress updates for readability.

OFF-to-voxel conversion and 3D CNN forward pass

Mesh-to-voxel and 3D CNN forward pass

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python run_voxel_demo.py --grid-size 32
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
Section 7.3 | mesh-to-voxel + CNN forward pass
Mesh fixture: data/ModelNet10/chair/train/chair_0001.off (synthetic closed cube, 12 triangles)
Voxel grid: (32, 32, 32); occupied cells: 24389/32768 (74.43%)
Tensor shape (B,C,D,H,W): (1, 1, 32, 32, 32); dtype=torch.float32
Device: cpu; CUDA available: False
Output logits shape: (1, 10) (10 classes)
Predicted class index (untrained demo head): 7
STATUS: PASS — conversion and forward shape checked; synthetic fixture only, no ModelNet10 accuracy claimed.
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme

MeshCNN edge feature and optimizer-step smoke run

MeshCNN edge-network smoke run

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python run_meshcnn_smoke.py --meshcnn-root /tmp/meshcnn-upstream
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
Section 7.4 | upstream MeshCNN CPU smoke test
MeshCNN source: /tmp/meshcnn-upstream
Fixture: data/meshcnn_fixture/icosphere.obj (generated trimesh icosphere; not SHREC)
Topology: vertices=42, triangles=80, edges=120
Edge feature tensor: (5, 120) (dihedral angle, opposite angles, ratios)
Input tensor: (1, 5, 120, 1); logits: (1, 30) (30-way SHREC-like head only)
Single synthetic-label cross-entropy: 3.177232; backward/Adam step: PASS
STATUS: PASS — upstream MeshConv/connectivity path exercised; no SHREC training or accuracy claimed.
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme

Appendix D - conversion execution

Terminal transcript screenshots are rendered on a black terminal theme from the saved command stdout/stderr logs in outputs/logs/. They preserve the actual run messages while removing transient carriage-return progress updates for readability.

Mesh / point cloud / voxel conversion results

Mesh / point cloud / voxel conversions

```
ganesh@ganeshlaptop:~/ganeshlaptop$ python run_conversions.py --grid-size 32
Deterministic smoke-test fixtures prepared (not ModelNet/SHREC data).
  Synthetic ModelNet40 layout: /home/user/bestwork/ganeshlaptop/data/synthetic_modelnet40_normal_resampled
  Train examples: 2 across 2 classes; test examples: 40 across 40 classes
  ModelNet10-style OFF mesh: /home/user/bestwork/ganeshlaptop/data/ModelNet10/chair/train/chair_0001.off
  MeshCNN triangular OBJ: /home/user/bestwork/ganeshlaptop/data/meshcnn_fixture/icosphere.obj
Section 7.5 | representation conversions
Mesh -> point cloud: data/ModelNet10/chair/train/chair_0001.off -> 4096 area-uniform surface points
  Saved: outputs/sampled_cube_4096.xyz; array shape=(4096, 3)
Point cloud -> mesh: SciPy ConvexHull -> 106 hull vertices, 208 triangular faces
  Method note: convex hull is used for a convex toy shape; it is not Poisson or Ball Pivoting.
Mesh -> voxel: occupancy shape=(32, 32, 32), active=24389 cells
Voxel -> point cloud: 24389 occupied-cell centers; saved outputs/voxel_occupied_centers.xyz
STATUS: PASS — all four conversions executed on a synthetic cube fixture.
Figure saved: outputs/figures/representation_conversion.png
EXIT_CODE=0
```

Saved stdout transcript · black terminal theme